

第二章 递归与分治

Chapter 2 Recursion and Divided-conquer

计算机学院 潘永生

递归的定义

在定义一个过程或函数时出现调用本过程或本函数的成分，称之为递归。若调用自身，称之为直接递归。若过程或函数p调用过程或函数q，而q又调用p，称之为间接递归。

任何间接递归都可以等价地转换为直接递归。

如果一个递归过程或递归函数中递归调用语句是最后一条执行语句，则称这种递归调用为尾递归。

【例2.1】设计求*n*!（*n*为正整数）的递归算法。

解：对应的递归函数如下：

```
int fun(int n)
{  if (n==1)                    //语句1
        return(1);              //语句2
    else                        //语句3
        return(fun(n-1)*n);     //语句4
}
```

在该函数fun(*n*)求解过程中，直接调用fun(*n*-1)（语句4）自身，所以它是一个直接递归函数。又由于递归调用是最后一条语句，所以它又属于尾递归。

一般来说，能够用递归解决的问题应该满足以下三个条件：

- 需要解决的问题可以转化为一个或多个子问题来求解，而这些子问题的求解方法与原问题完全相同，只是在数量规模上不同。
- 递归调用的次数必须是有限的。
- 必须有结束递归的条件来终止递归。

何时使用递归

在以下三种情况下，常常要用到递归的方法。

1. 定义是递归的

有许多数学公式、数列等的定义是递归的。例如，求n!和Fibonacci数列等。这些问题的求解过程可以将其递归定义直接转化为对应的递归算法。

数据结构是递归的

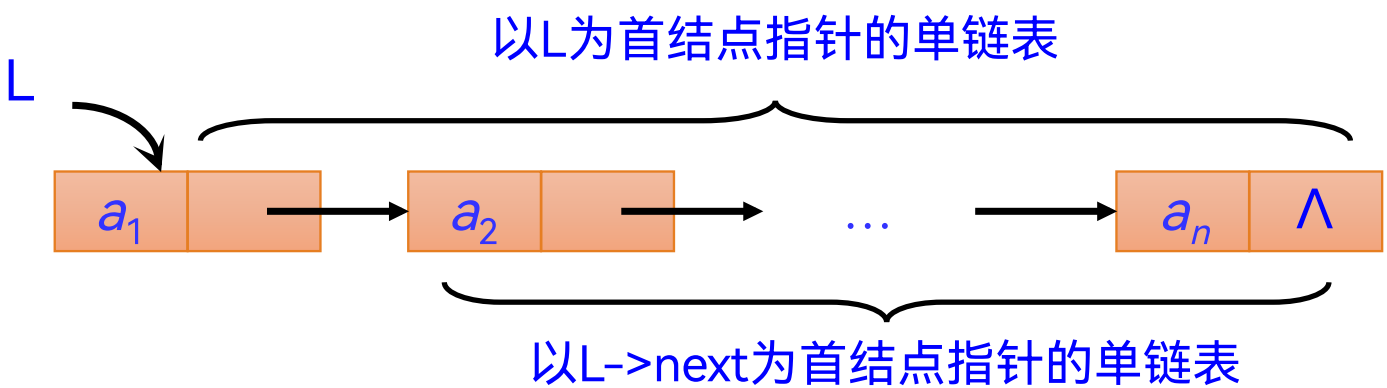
有些数据结构是递归的。例如单链表就是一种递归数据结构，其结点类型声明如下：

```
typedef struct LNode
{   ElemType data;
    struct LNode *next;
} LinkList;
```

为什么可以这样递归定义类型

结构体LNode的定义中用到了它自身，即指针域next是一种指向自身类型的指针，所以它是一种递归数据结构。

不带头结点单链表示意图



体现出数据结构的递归性。

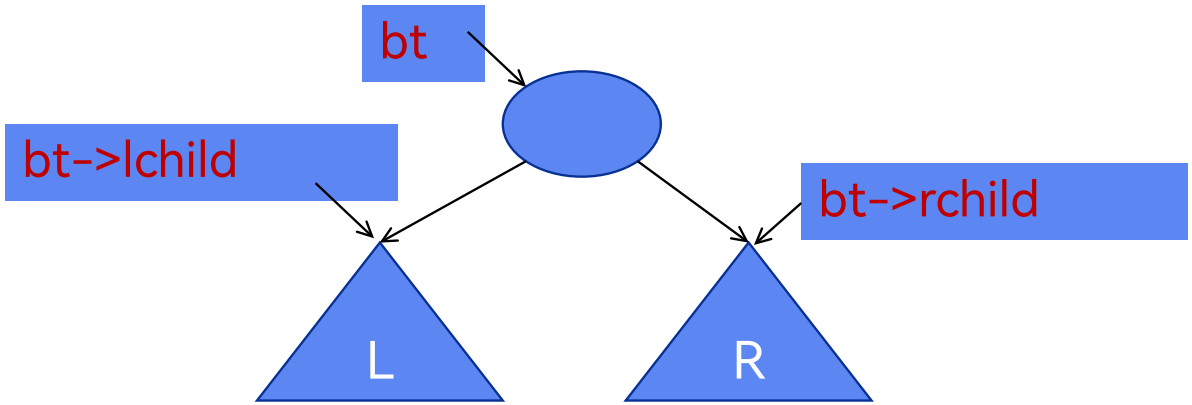
对于递归数据结构，采用递归的方法编写算法既方便又有效。例如，求一个不带头结点的单链表L的所有data域（假设为int型）之和的递归算法如下：

```
int Sum(LinkList *L)
{   if (L==NULL)
            return 0;
      else
            return(L->data+Sum(L->next));
}
```

【例2.2】分析二叉树的二叉链存储结构的递归性，设计求非空二叉链bt中所有结点值之和的递归算法，假设二叉链的data域为int型。

解：二叉树采用二叉链存储结构，其结点类型定义如下：

```
typedef struct BNode
{   int data;
    struct BNode *lchild，*rchild;
} BTNode;              //二叉链结点类型
```




```
int Sumbt(BTNode *bt)              //求二叉树bt中所有结点值之和
{  if (bt->lchild==NULL && bt->rchild==NULL)
      return bt->data;    //只有一个结点时返回该结点值
   else                           //否则返回左、右子树结点值之和加上根结点值
      return Sumbt(bt->lchild)+ Sumbt(bt->rchild)+bt->data);
}
```

3. 问题的求解方法是递归的

有些问题的解法是递归的，典型的有Hanoi问题求解。



盘片移动时必须遵守以下规则：每次只能移动一个盘片；盘片可以插在X、Y和Z中任一塔座；任何时候都不能将一个较大的盘片放在较小的盘片上。

设计递归求解算法，并将其转换为非递归算法。

设Hanoi(*n*，*x*，*y*，*z*)表示将*n*个盘片从*x*通过*y*移动到*z*上，递归分解的过程是：



- Characteristics of recursion
  - **Every recursive process consists of two parts**
    - A smallest, base case that is processed without recursion; and
    - A general method that reduces a particular case to one or more of the smaller cases, thereby making progress toward eventually reducing the problem all the way to the base case.

- Designing recursive algorithm
  - **Outline your algorithm**.

    Combine the stopping rule and the key step, using an if statement to select between them.

```
if (simplest case) then
    solve directly
else
    make recursive call to a simpler
case
```

- Designing recursive algorithm
  - **Find the key step**.
    Begin by asking yourself , “How can this problem be divided into parts?” or “How will the key step in the middle be done?”
  - **Find a stopping rule.**
    This stopping rule is usually the small, special case that is trivial or easy to handle without recursion.

- Designing recursive algorithm
  - **Check termination.**

    Verify that the recursion will always terminate. Be sure that your algorithm correctly handles extreme cases.
  - **Draw a recursion tree.**

    The height of the tree is closely related to the amount of memory that the program will require, and the total size of the tree reflects the number of times the key step will be done.

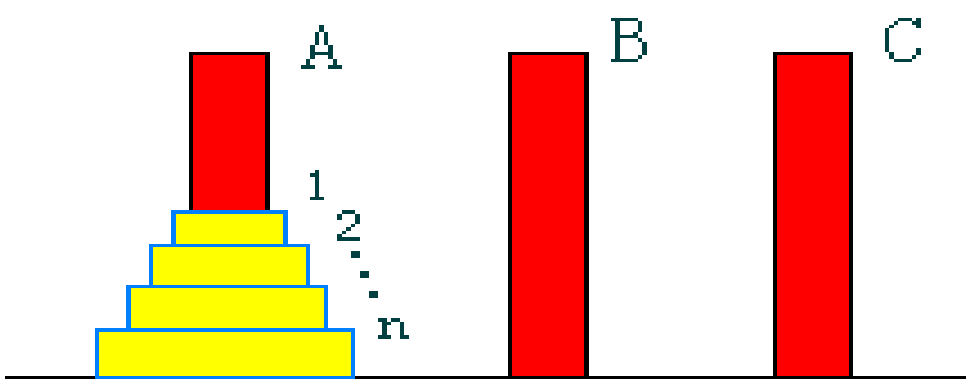
Hanoi塔问题

设a,b,c是3个塔座。开始时，在塔座a上有一叠共n个圆盘，这些圆盘自下而上，由大到小地叠在一起。各圆盘从小到大编号为1,2,...,n,现要求将塔座a上的这一叠圆盘移到塔座b上，并仍按同样顺序叠置。在移动圆盘时应遵守以下移动规则：

规则1：每次只能移动1个圆盘；

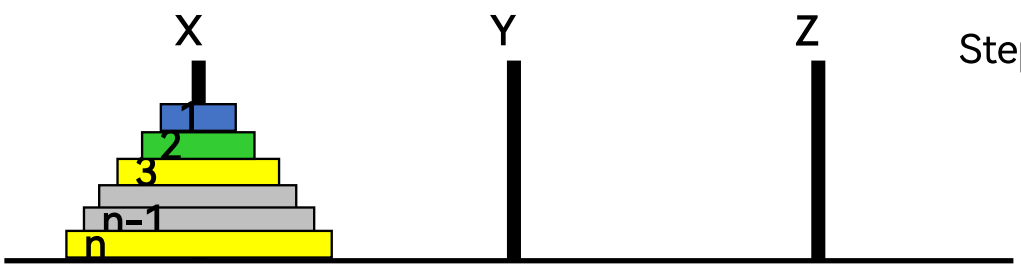
规则2：任何时刻都不允许将较大的圆盘压在较小的圆盘之上；

规则3：在满足移动规则1和2的前提下，可将圆盘移至a,b,c中任一塔座上。

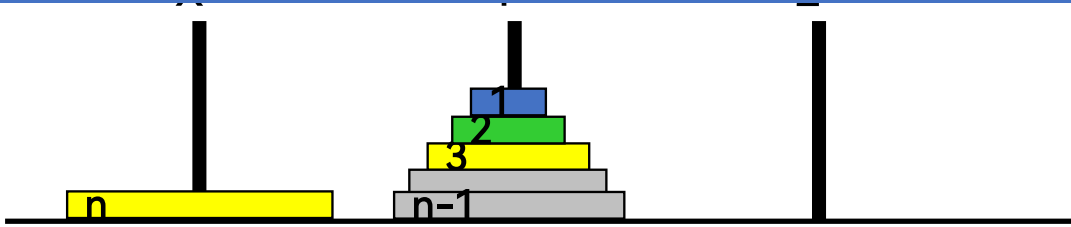


2.2 Hanoi塔问题

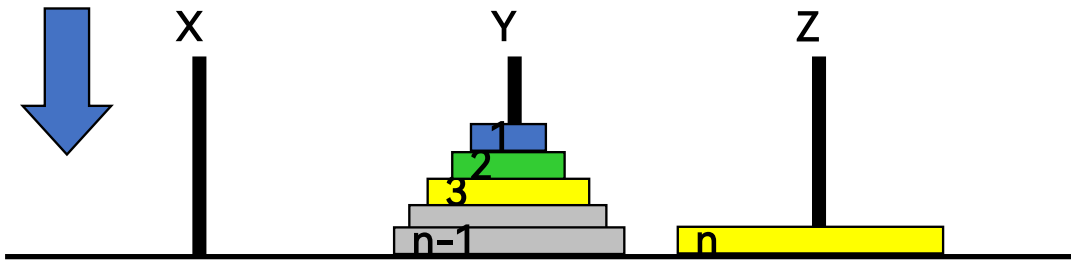
Divide & conquer



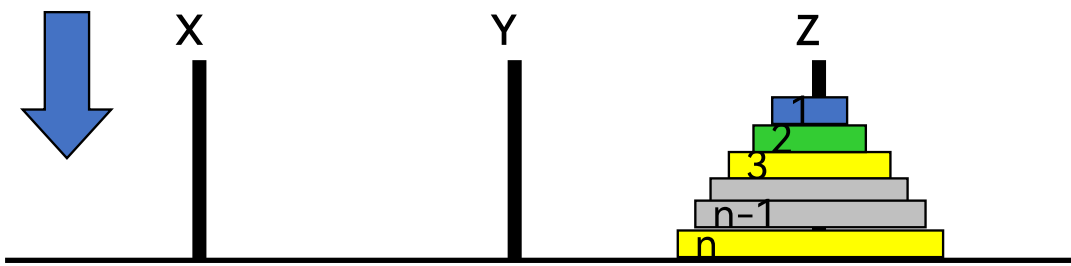
Step1



Step2



Step3



Demo

Method specification

```
void move(int count, int start, int finish, int temp);
```

Pre: There are at least count disks on the tower start. The top disk (if any) on each of towers temp and finish is larger than any of the top count disks on tower start.

Post: The top count disks on start have been moved to finish; temp (used for temporary storage) has been returned to its starting position.

Program for Hanoi

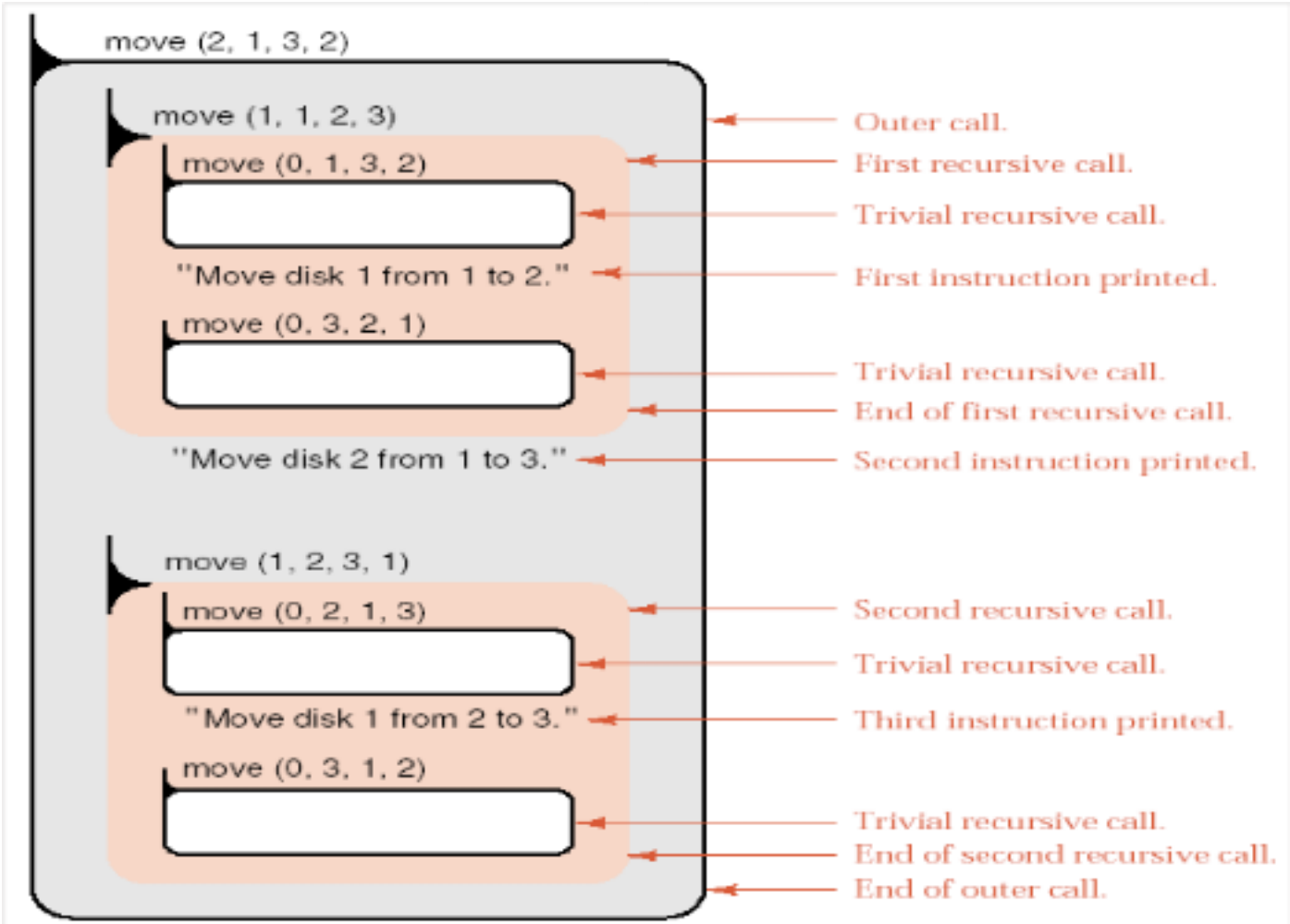
```
const int disks = 64;      // Make this constant much smaller
          // to run program.
void move(int count, int start, int finish, int temp);
/*
Pre:  None.
Post: The simulation of the Towers of Hanoi has terminated.
*/
main()
{
   move(disks, 1, 3, 2);
}
```

Hanoi recursive function

```
void move(int count, int start, int finish, int temp)
1:{
2:    if (count > 0) {
3:      move(count - 1, start, temp, finish);
4:      cout << "Move disk " << count << " from " <<                    start
<< " to " << finish << "." << endl;
5:      move(count - 1, temp, finish, start);
6:   }
7:}
```

Example:

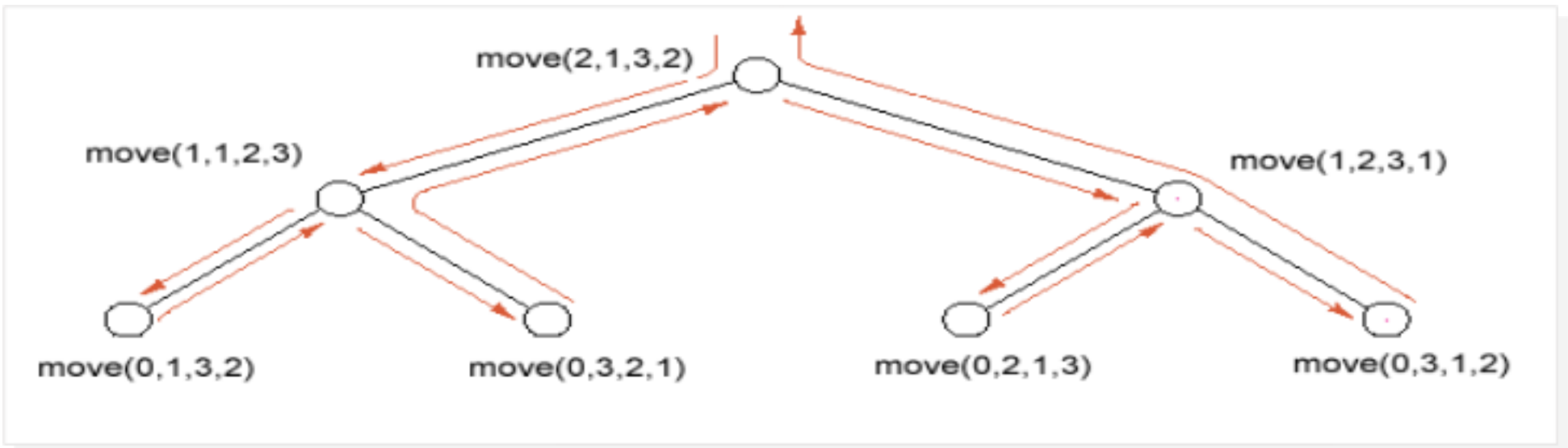
move(2, 1, 3, 2)



Recursion tree of Hanoi

Example:

move(2, 1, 3, 2)

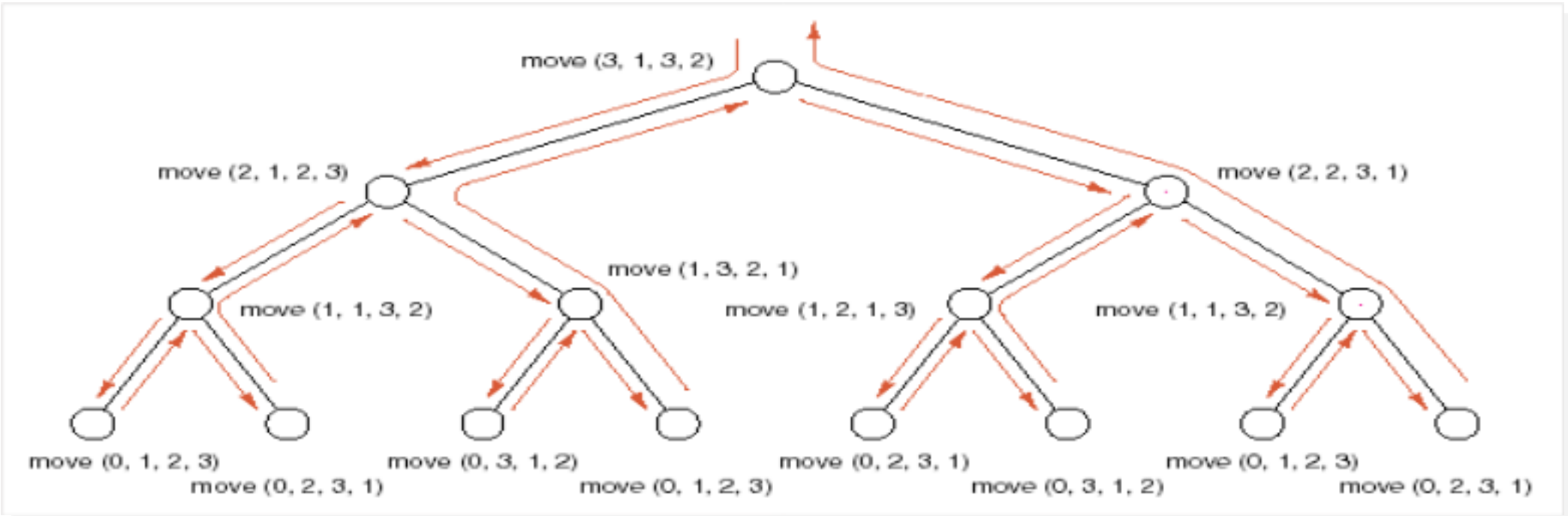


Recursion and stack

Recursion tree of Hanoi

Example:

move(3, 1, 3, 2)



优点： 结构清晰，可读性强，而且容易用数学归纳法来证明算法的正确性，因此它为设计算法、调试程序带来很大方便。

缺点： 递归算法的运行效率较低，无论是耗费的计算时间还是占用的存储空间都比非递归算法要多。

对于一个规模为n的问题：若该问题可以容易地解决（比如说规模n较小）则直接解决，否则将其分解为k个规模较小的子问题，这些子问题互相独立且与原问题形式相同，递归地解这些子问题，然后将各子问题的解合并得到原问题的解。

这种算法设计策略叫做分治法。

分治法所能解决的问题一般具有以下几个特征：

（1）该问题的规模缩小到一定的程度就可以容易地解决。

（2）该问题可以分解为若干个规模较小的相同问题。

（3）利用该问题分解出的子问题的解可以合并为该问题的解。

（4）该问题所分解出的各个子问题是相互独立的，即子问题之间不包含公共的子问题。

分治法通常采用递归算法设计技术，在每一层递归上都有3个步骤：

① 分解：将原问题分解为若干个规模较小，相互独立，与原问题形式相同的子问题。

② 求解子问题：若子问题规模较小而容易被解决则直接求解，否则递归地求解各个子问题。

③ 合并：将各个子问题的解合并为原问题的解。

分治法的一般的算法设计框架如下：

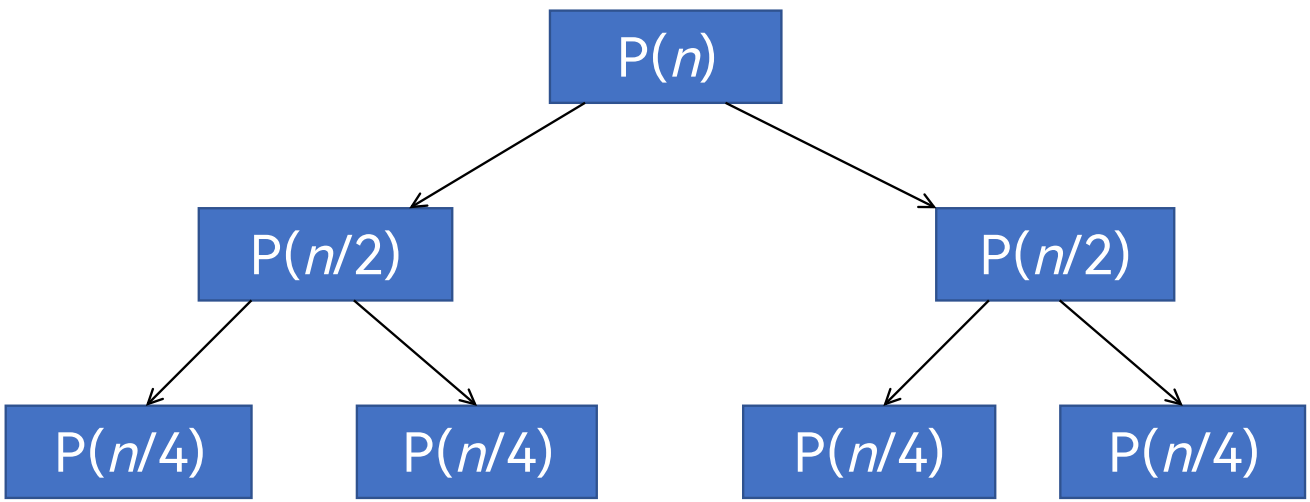
```
divide-and-conquer(P)
{  if |P|≤n0 return adhoc(P);
   将P分解为较小的子问题 P1，P2，…，Pk;
   for(i=1;i<=k;i++)                    //循环处理k次
     yi=divide-and-conquer(Pi);    //递归解决Pi
      return merge(y1，y2，…，yk);    //合并子问题
}
```

根据分治法的分割原则，原问题应该分为多少个子问题才较适宜？各个子问题的规模应该怎样才为适当？

这些问题很难予以肯定的回答。但人们从大量实践中发现，在用分治法设计算法时，最好使子问题的规模大致相同。换句话说，将一个问题分成大小相等的k个子问题的处理方法是行之有效的。

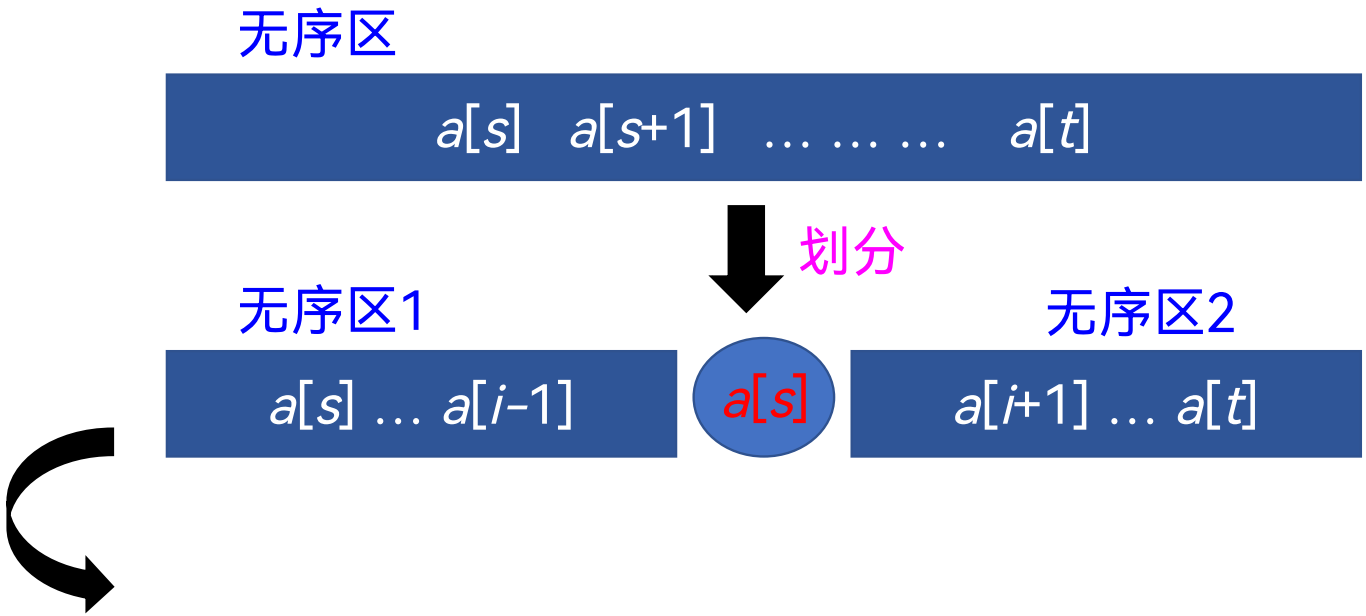
当k=1时称为减治法。

许多问题可以取 k=2，称为二分法，如图4.1所示，这种使子问题规模大致相等的做法是出自一种平衡子问题的思想，它几乎总是比子问题规模不等的做法要好。



基本思想：在待排序的n个元素中任取一个元素（通常取第一个元素）作为基准，把该元素放入最终位置后，整个数据序列被基准分割成两个子序列，所有小于基准的元素放置在前子序列中，所有大于基准的元素放置在后子序列中，并把基准排在这两个子序列的中间，这个过程称作划分。

然后对两个子序列分别重复上述过程，直至每个子序列内只有一个记录或空为止。



f(*a*,*s*,*t*) ≡ 不做任何事情　　当*a*[*s*..*t*]中长度小于2

f(*a*,*s*,*t*) ≡ *i*=Partition(*a*,s,*t*);　　其他情况

f(*a*,*s*,*i*-1);

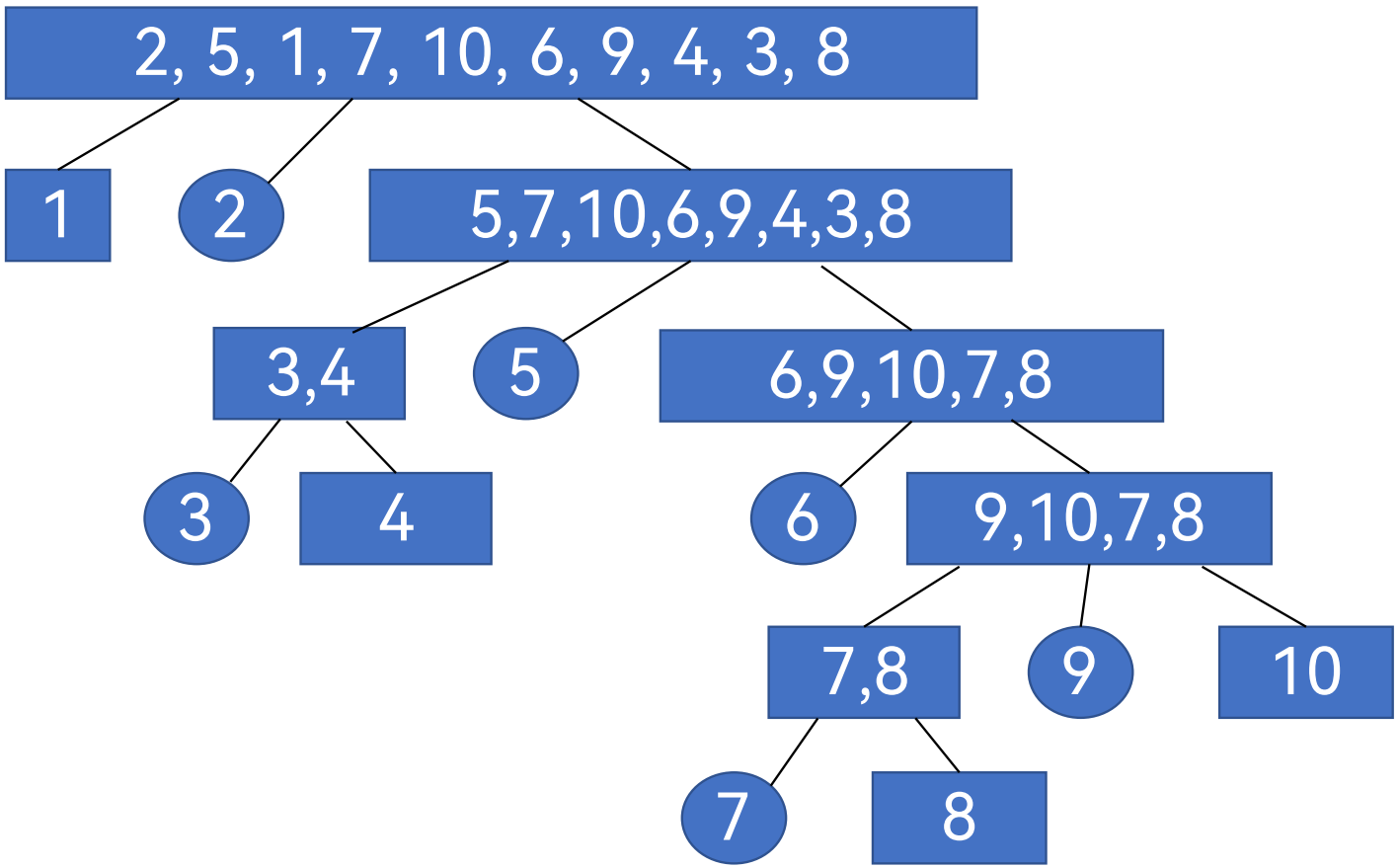
f(*a*,*i*,*t*);

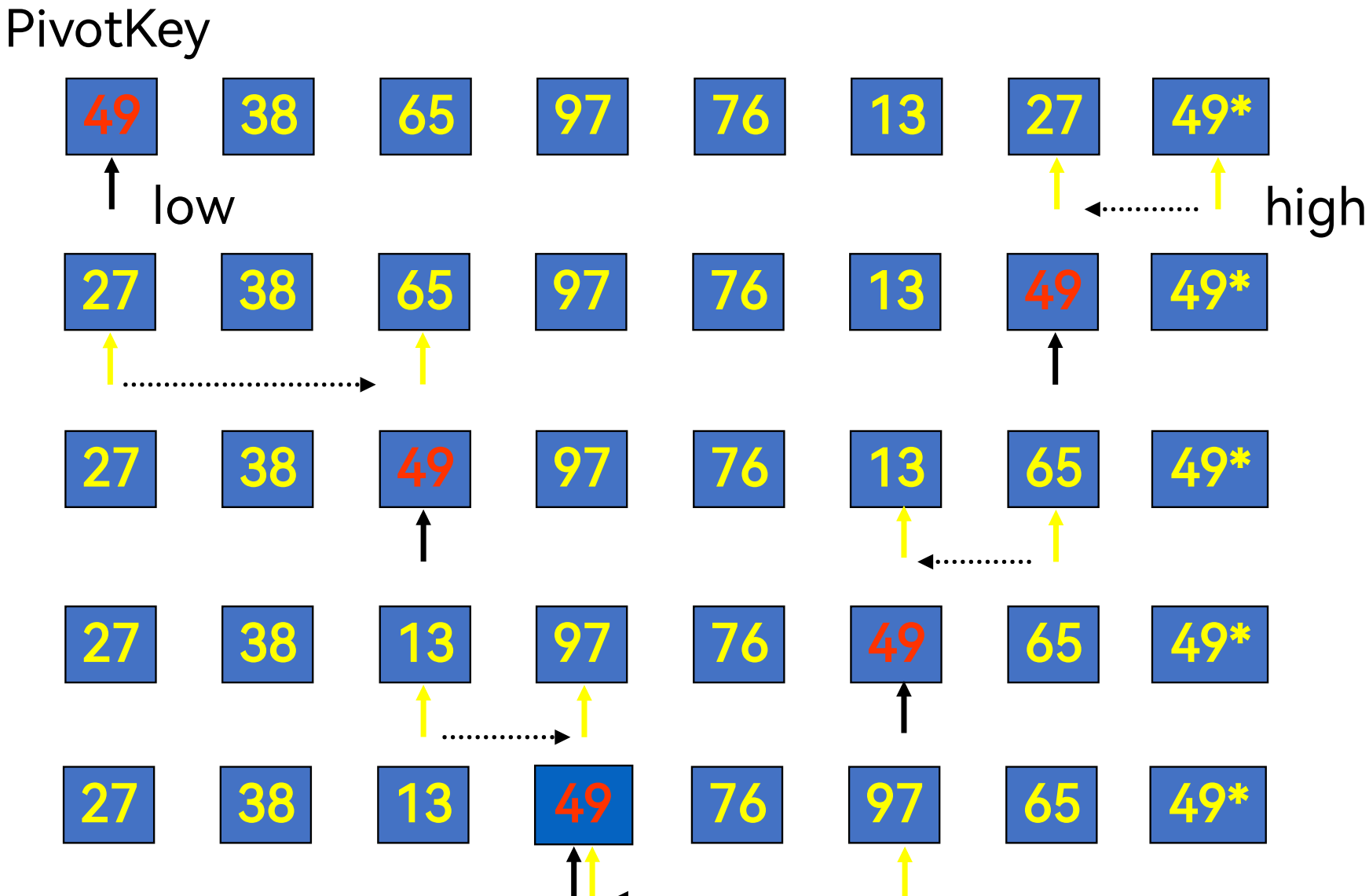
① 分解：将原序列$a[s..t]$分解成两个子序列$a[s..i-1]$和$a[i+1..t]$，其中i为划分的基准位置。

② 求解子问题：若子序列的长度为0或为1，则它是有序的，直接返回；否则递归地求解各个子问题。

③ 合并：由于整个序列存放在数组中a中，排序过程是就地进行的，合并步骤不需要执行任何操作。

例如，对于{2，5，1，7，10，6，9，4，3，8}序列，其快速排序过程如下图所示。




PivotKey
49
38
65
97
76
13
27
49*
low
high
27
38
65
97
76
13
49
49*
27
38
49
97
76
13
65
49*
27
38
13
97
76
49
65
49*
27
38
13
49
76
97
65
49*

```
int Partition(int a[], int s, int t)  //划分算法
{  int i=s, j=t;
   int tmp=a[s];  //用序列的第1个记录作为基准

   while (i!=j)      //从序列两端交替向中间扫描，直至i=j为止
   {  while (j>i && a[j]>=tmp)
                  j--;      //从右向左扫描，找第1个关键字小于tmp的a[j]
      a[i]=a[j];     //将a[j]前移到a[i]的位置
      while (i<j && a[i]<=tmp)
                  i++;      //从左向右扫描，找第1个关键字大于tmp的a[i]
      a[j]=a[i];     //将a[i]后移到a[j]的位置
      }

      a[i]=tmp;
      return i;
}
```

快速排序算法：

```
void QuickSort(int a[]，int s，int t)
//对a[s..t]元素序列进行递增排序
{ if (s<t)                    //序列内至少存在2个元素的情况
  {  int i=Partition(a，s，t);
     QuickSort(a，s，i-1);     //对左子序列递归排序
     QuickSort(a，i+1，t);     //对右子序列递归排序
  }
}
```

【算法分析】快速排序的时间主要耗费在划分操作上，对长度为n的区间进行划分，共需n-1次关键字的比较，时间复杂度为O(n)。

对n个记录进行快速排序的过程构成一棵递归树，在这样的递归树中，每一层至多对n个记录进行划分，所花时间为O(n)。

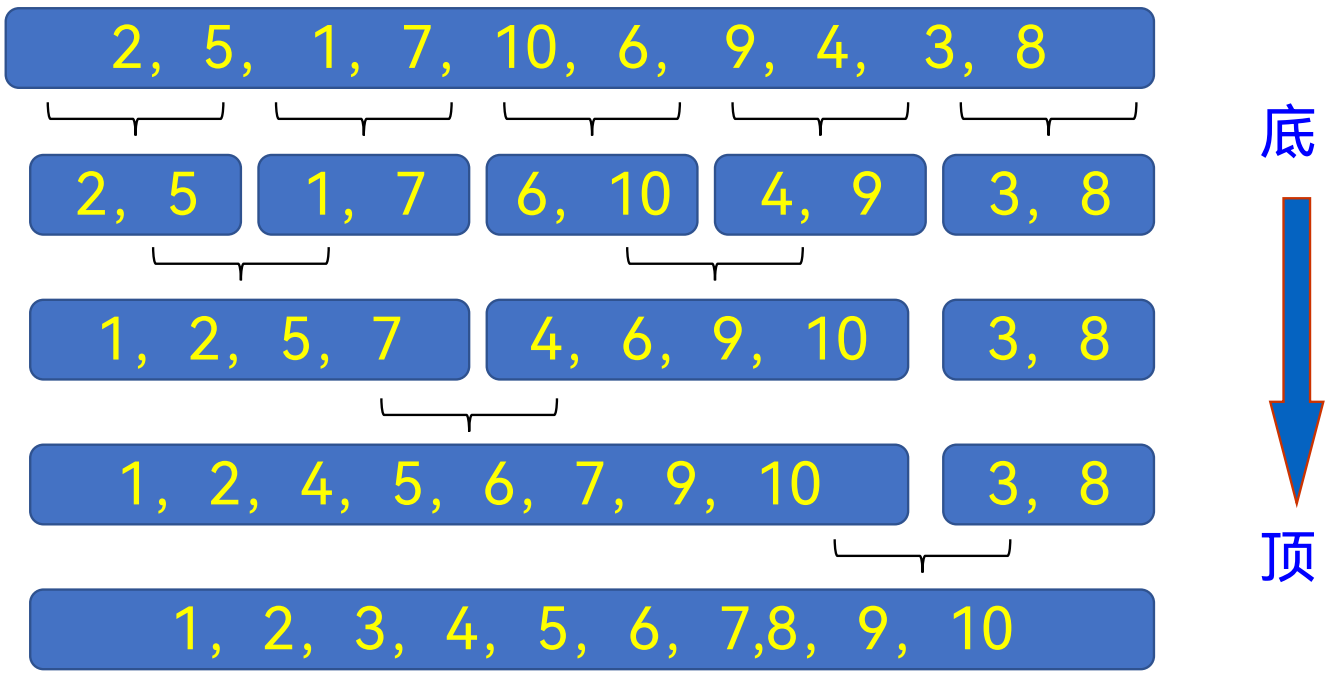
当初始排序数据正序或反序时，此时的递归树高度为n，快速排序呈现最坏情况，即最坏情况下的时间复杂度为O(n^2)；

当初始排序数据随机分布，使每次分成的两个子区间中的记录个数大致相等，此时的递归树高度为$\log_2 n$，快速排序呈现最好情况，即最好情况下的时间复杂度为O($n\log_2 n$)。快速排序算法的平均时间复杂度也是O($n\log_2 n$)。

归并排序的基本思想是：首先将$a[0..n-1]$看成是n个长度为1的有序表，将相邻的k（$k\geq 2$）个有序子表成对归并，得到n/k个长度为k的有序子表；然后再将这些有序子表继续归并，得到n/k^2个长度为k^2的有序子表，如此反复进行下去，最后得到一个长度为n的有序表。

若$k=2$，即归并在相邻的两个有序子表中进行的，称为二路归并排序。若$k>2$，即归并操作在相邻的多个有序子表中进行，则叫多路归并排序。

例如，对于{2，5，1，7，10，6，9，4，3，8}序列，其排序过程如下图所示，图中方括号内是一个有序子序列。



二路归并排序的分治策略如下：

循环$\lceil \log_2 n \rceil$次，length依次取1、2、…、$\log_2 n$。每次执行以下步骤：

① 分解：将原序列分解成length长度的若干子序列。

② 求解子问题：将相邻的两个子序列调用Merge算法合并成一个有序子序列。

③ 合并

```
void Merge(int a[]，int low，int mid，int high)
//a[low..mid]和a[mid+1..high]→a[low..high]
{  int *tmpa;
   int i=low，j=mid+1，k=0;
   tmpa=(int *)malloc((high-low+1)*sizeof(int));//临时空间
   while (i<=mid && j<=high)
     if (a[i]<=a[j])        //将第1子表中的元素放入tmpa中
         {  tmpa[k]=a[i];  i++; k++; }
         else                    //将第2子表中的元素放入tmpa中
         {  tmpa[k]=a[j];   j++; k++; }
   while (i<=mid)        //将第1子表余下部分复制到tmpa
   {  tmpa[k]=a[i]; i++; k++; }
   while (j<=high)             //将第2子表余下部分复制到tmpa
   {  tmpa[k]=a[j]; j++; k++; }
   for (k=0，i=low;i<=high;k++，i++)  //将tmpa复制回a中
        a[i]=tmpa[k];
   free(tmpa);                 //释放tmpa所占内存空间
}
```

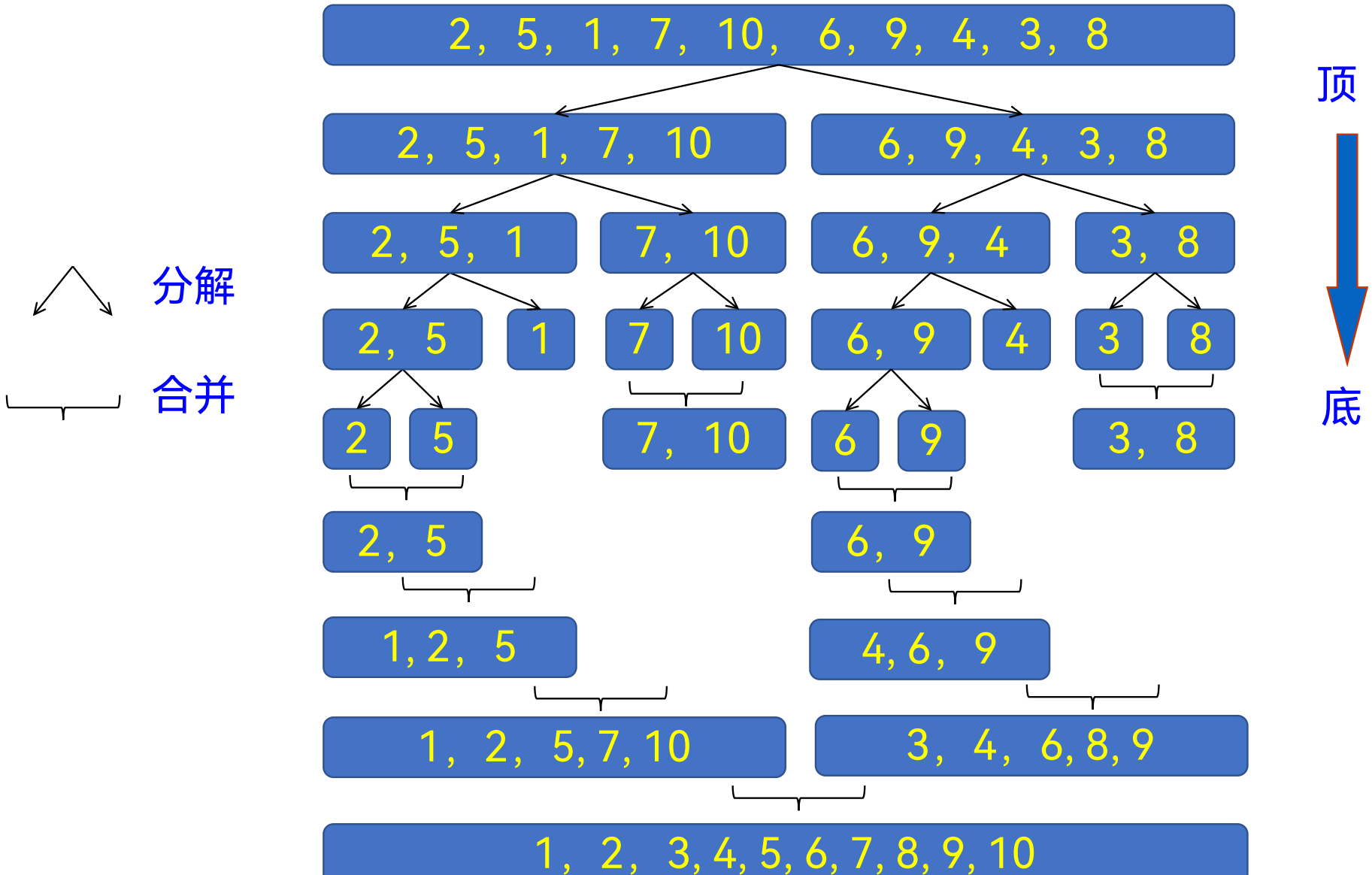
```
void MergePass(int a[]，int length，int n)
//一趟二路归并排序
{  int i;
   for (i=0;i+2*length-1<n;i=i+2*length)  //归并length长的两相邻子表
            Merge(a，i，i+length-1，i+2*length-1);

   if (i+length-1<n)                    //余下两个子表，后者长度小于length
          Merge(a，i，i+length-1，n-1);    //归并这两个子表
}
```

```
void MergeSort(int a[]，int n)     //二路归并算法
{  int length;
   for (length=1;length<n;length=2*length)
            MergePass(a，length，n);
}
```

【算法分析】对于上述二路归并排序算法，当有n个元素时，需要$\lceil \log_2 n \rceil$趟归并，每一趟归并，其元素比较次数不超过$n-1$，元素移动次数都是n，因此归并排序的时间复杂度为$O(n\log_2 n)$。

例如，对于{2，5，1，7，10，6，9，4，3，8}序列，说明其自顶向下的二路归并排序的过程。


分解
合并
2，5，1，7，10，6，9，4，3，8
2，5，1，7，10
6，9，4，3，8
2，5，1
7，10
6，9，4
3，8
2，5
1
7
10
6，9
4
3
8
2
5
7，10
6
9
3，8
2，5
6，9
1，2，5
4，6，9
1，2，5，7，10
3，4，6，8，9
1，2，3，4，5，6，7，8，9，10
顶
底

设归并排序的当前区间是*a*[low..high]，则递归归并的两个步骤如下：

① 分解：将序列*a*[low..high]一分为二，即求mid=(low+high)/2；递归地对两个子序列*a*[low..mid]和*a*[mid+1..high]进行继续分解。其终结条件是子序列长度为1（因为一个元素的子表一定是有序表）。

② 合并：与分解过程相反，将已排序的两个子序列*a*[low..mid]和*a*[mid+1..high]归并为一个有序序列*a*[low..high]。

对应的二路归并排序算法如下：

```
void MergeSort(int a[]，int low，int high)
//二路归并算法
{  int mid;
     if (low<high)              //子序列有两个或以上元素
     {    mid=(low+high)/2;          //取中间位置
          MergeSort(a，low，mid);//对a[low..mid]子序列排序
          MergeSort(a，mid+1，high); //对a[mid+1..high]子序列排序
          Merge(a，low，mid，high);  //将两子序列合并，见前面的算法
     }
}
```

递归出口为序列长度为1或者0!

【算法分析】设MergeSort(a，0，n-1)算法的执行时间为$T(n)$，显然Merge(a，0，$n/2$，n-1)的执行时间为O(n)，所以得到以下递推式：

$$T(n)=1 \quad \text{当} n=1$$
$$T(n)=2T(n/2)+O(n) \quad \text{当} n>1$$

容易推出，$T(n)=O(n\log_2 n)$。

【问题描述】对于给定的含有n元素的无序序列，求这个序列中最大和次大的两个不同的元素。

例如：（2, 5, 1, 4, 6, 3），最大元素为6，次大元素为5。

2.6 查找最大和次大元素

【问题求解】对于无序序列*a*[low.high]中，采用分治法求最大元素max1和次大元素max2的过程如下：

（1）*a*[low.high]中只有一个元素：则max1=a[low]，max2=-INF（-∞）（要求它们是不同的元素）。

（2）*a*[low.high]中只有两个元素：则max1=MAX{a[low]，a[high]}，max2=MIN{a[low]，a[high]}。

（3）*a*[low.high]中有两个以上元素：按中间位置mid=(low+high)/2划分为a[low..mid]和a[mid+1..high]左右两个区间（注意左区间包含a[mid]元素）。

求出左区间最大元素lmax1和次大元素lmax2，求出右区间最大元素rmax1和次大元素rmax2。

合并：若lmax1>rmax1，则max1=lmax1，max2=MAX{lmax2，rmax1}；否则max1=rmax1，max2=MAX{lmax1，rmax2}。

2. 6 查找最大和次大元素

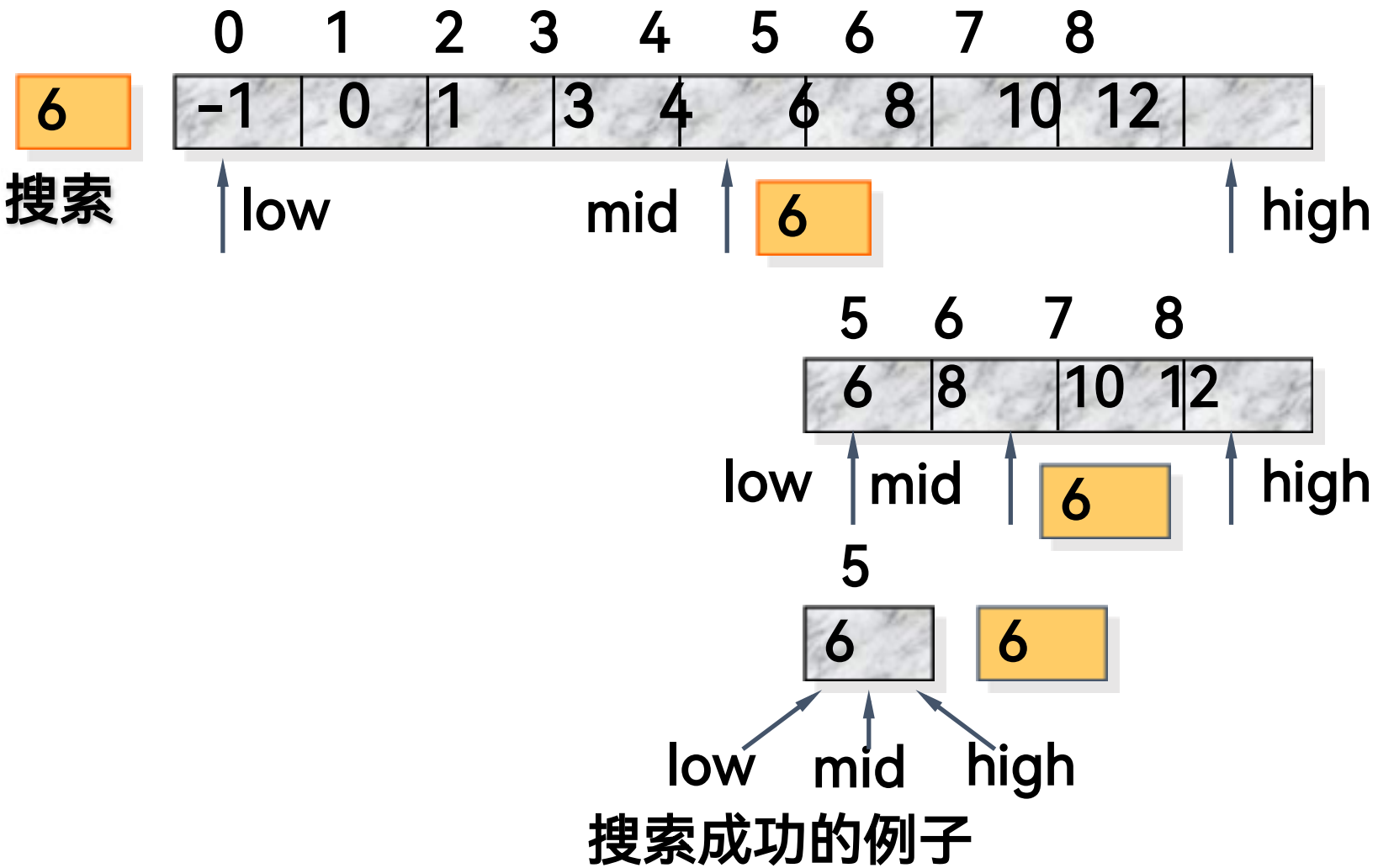

```
void solve(int a[],int low,int high,int &max1,int &max2)
{   if (low==high)                  //区间只有一个元素
    {   max1=a[low];    max2=-INF;  }
    else if (low==high-1)       //区间只有两个元素
    {   max1=max(a[low],a[high]); max2=min(a[low],a[high]); }
    else                        //区间有两个以上元素
    {   int mid=(low+high)/2;
        int lmax1,lmax2;
        solve(a,low,mid,lmax1,lmax2);         //左区间求lmax1和lmax2
        int rmax1,rmax2;
        solve(a,mid+1,high,rmax1,rmax2);  //右区间求lmax1和lmax2
        if (lmax1>rmax1)
        {   max1=lmax1;
            max2=max(lmax2,rmax1);      //lmax2,rmax1中求次大元素
        }
        else
        {   max1=rmax1;
            max2=max(lmax1,rmax2);      //lmax1,rmax2中求次大元素
        }
    }
}
```

【算法分析】对于solve(*a*，0，*n*-1，max1，max2)调用，其比较次数的递推式为：

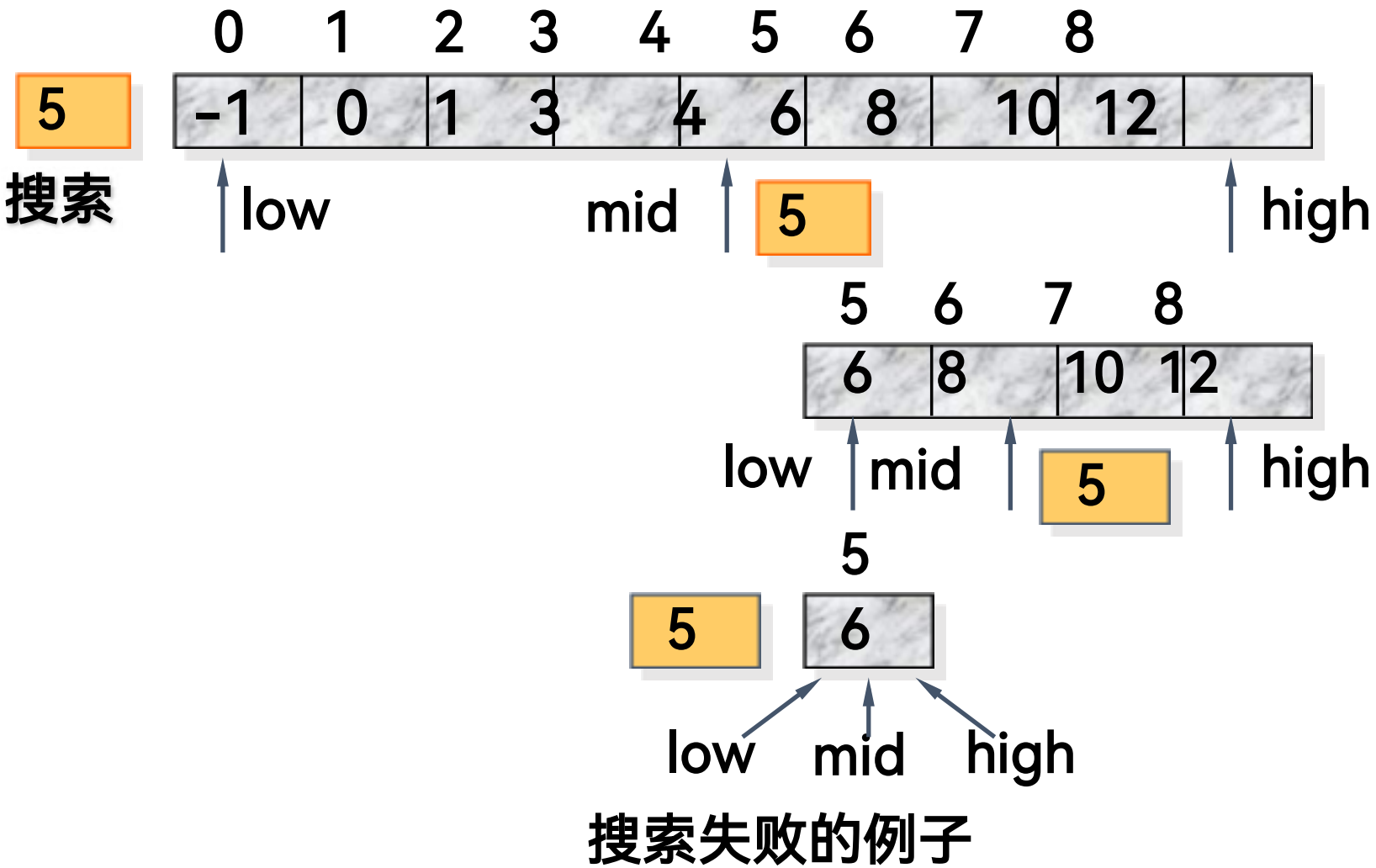
$T(1)=T(2)=1$

$T(n)=2T(n/2)+1$　//合并的时间为O(1)

可以推导出$T(n)=O(n)$。


0 1 2 3 4 5 6 7 8
6
-1 0 1 3 4 6 8 10 12
搜索
low
mid
6
high
5 6 7 8
6 8 10 12
low
mid
6
high
5
6
6
low
mid
high

搜索成功的例子


0 1 2 3 4 5 6 7 8
5
-1 0 1 3 4 6 8 10 12
搜索
low
mid
5
high
5 6 7 8
6 8 10 12
low
mid
5
high
5
5
6
low
mid
high

搜索失败的例子

基本思路：设a[low..high]是当前的查找区间，首先确定该区间的中点位置mid=⌊(low+high)/2⌋；然后将待查的k值与a[mid].key比较：

（1）若k==a[mid]，则查找成功并返回该元素的物理下标；

（2）若k<a[mid]，则由表的有序性可知a[mid..high]均大于k，因此若表中存在关键字等于k的元素，则该元素必定位于左子表a[low..mid-1]中，故新的查找区间是左子表a[low..mid-1]；

（3）若k>a[mid]，则要查找的k必在位于右子表a[mid+1..high]中，即新的查找区间是右子表a[mid+1..high]。

下一次查找是针对新的查找区间进行的。

```
int BinSearch(int a[]，int low，int high，int k)
//拆半查找算法
{ int mid;
  if (low<=high)        //当前区间存在元素时
  {  mid=(low+high)/2;  //求查找区间的中间位置
        if (a[mid]==k)          //找到后返回其物理下标mid
            return mid;
        if (a[mid]>k)           //当a[mid]>k时
            return BinSearch(a，low，mid-1，k);
        else                    //当a[mid]<k时
            return BinSearch(a，mid+1，high，k);
  }
   else return -1;              //若当前查找区间没有元素时返回-1
}
```

【算法分析】折半查找算法的主要时间花费在元素比较上，对于含有n个元素的有序表，采用折半查找时最坏情况下的元素比较次数为$C(n)$，则有：

$$C(n)=1 \quad \text{当} n=1$$

$$C(n)\le 1+C(\lfloor n/2 \rfloor) \quad \text{当} n\ge 2$$

由此得到：$C(n)\le \lfloor \log_2 n \rfloor +1$

折半查找的主要时间花在元素比较上，所以算法的时间复杂度为$O(\log_2 n)$。

【问题描述】对于给定的含有n元素的无序序列，求这个序列中第k（$1 \le k \le n$）小的元素。

【问题求解】假设无序序列存放在$a[0..n-1]$中，若将a递增排序，则第k小的元素为$a[k-1]$。

采用类似于快速排序的思想。

该元素的下标为$k-1$

↕

对于序列$a[s..t]$，在其中查找第k小元素的过程如下：

将$a[s]$作为基准划分，其对应下标为i。3种情况：

- 若$k-1=i$，$a[i]$即为所求，返回$a[i]$。
- 若$k-1<i$，第k小的元素应在$a[s..i-1]$子序列中，递归在该子序列中求解并返回其结果。
- 若$k-1>i$，第k小的元素应在$a[i+1..t]$子序列中，递归在该子序列中求解并返回其结果。

2.8 寻找一个序列中第k小元素


```
int QuickSelect(int a[], int s, int t, int k)
//在a[s..t]序列中找第k小的元素
{  int i=s, j=t,tmp;
   if (s<t)
   {  tmp=a[s];
      while (i!=j)            //从区间两端交替向中间扫描，直至i=j为止
      {  while (j>i && a[j]>=tmp) j--;
                 a[i]=a[j];          //将a[j]前移到a[i]的位置
                 while (i<j && a[i]<=tmp) i++;
                 a[j]=a[i];          //将a[i]后移到a[j]的位置
      }
      a[i]=tmp;

      if (k-1==i) return a[i];
      else if (k-1<i) return QuickSelect(a, s, i-1, k);
                              //在左区间中递归查找
      else return QuickSelect(a, i+1, t, k);
                              //在右区间中递归查找
   }
   else if (s==t && s==k-1)  //区间内只有一个元素且为a[k-1]
      return a[k-1];
}
```

【算法分析】对于QuickSelect(*a*，*s*，*t*，*k*)算法，设序列*a*中含有*n*个元素，其比较次数的递推式为：

$T(n)=T(n/2)+O(n)$

可以推导出$T(n)=O(n)$，这是最好的情况，即每次划分的基准恰好是中位数，将一个序列划分为长度大致相等的两个子序列。

在最坏情况下，每次划分的基准恰好是序列中的最大值或最小值，则处理区间只比上一次减少1个元素，此时比较次数为$O(n^2)$。

在平均情况下该算法的时间复杂度为$O(n)$。

【问题描述】对于一个长度为n的有序序列（假设均为升序序列）$a[0..n-1]$，处于中间位置的元素称为a的中位数。

设计一个算法求给定的两个有序序列的中位数。

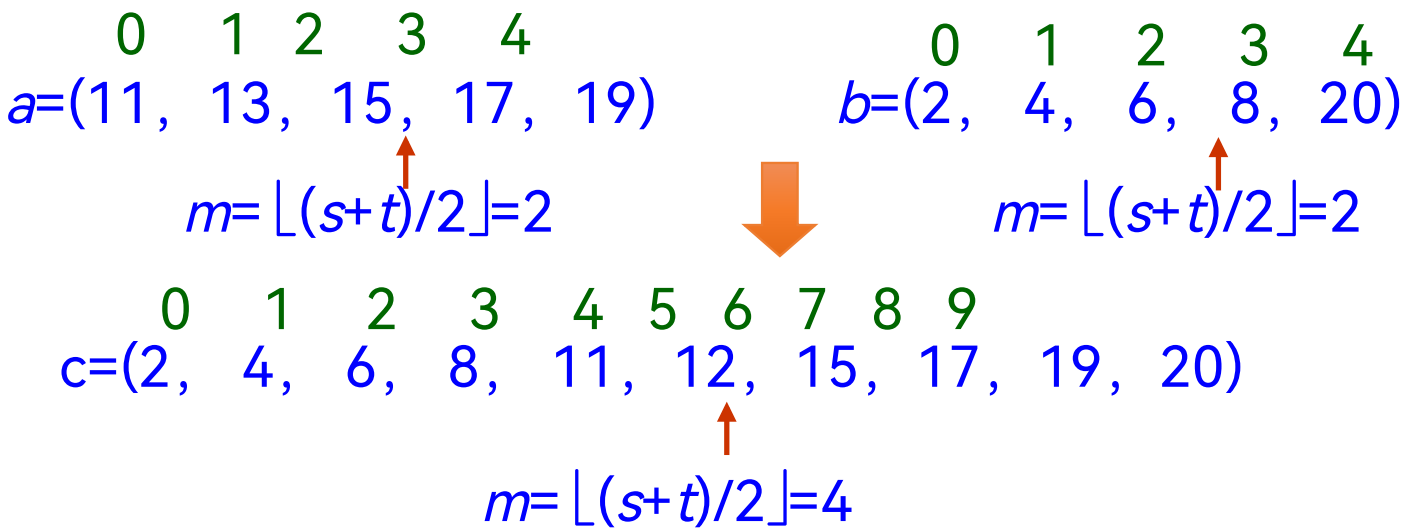
例如，若序列a=(11，13，15，17，19)，其中位数是15，若b=(2，4，6，8，20)，其中位数为6。两个等长有序序列的中位数是含它们所有元素的有序序列的中位数，例如a、b两个有序序列的中位数为11。

a=(11，13，15，17，19)　　b=(2，4，6，8，20)



c=(2，4，6，8，11，13，15，17，19，20)

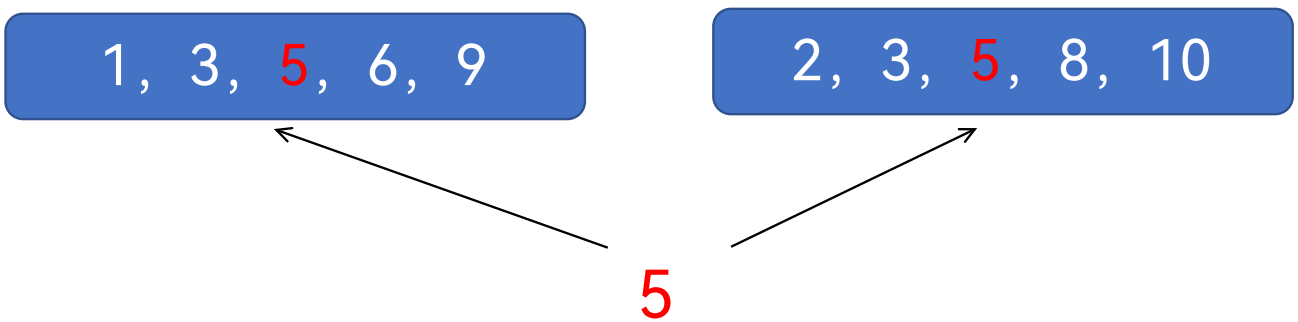
【问题求解】对于含有n个元素的有序序列$a[s..t]$，当n为奇数时，中位数是出现在$m=\lfloor(s+t)/2\rfloor$处；当n为偶数时，中位数下标有$m=\lfloor(s+t)/2\rfloor$（下中位）和$m=\lfloor(s+t)/2\rfloor+1$（上中位）两个。为了简单，仅考虑中位数为$m=\lfloor(s+t)/2\rfloor$处。



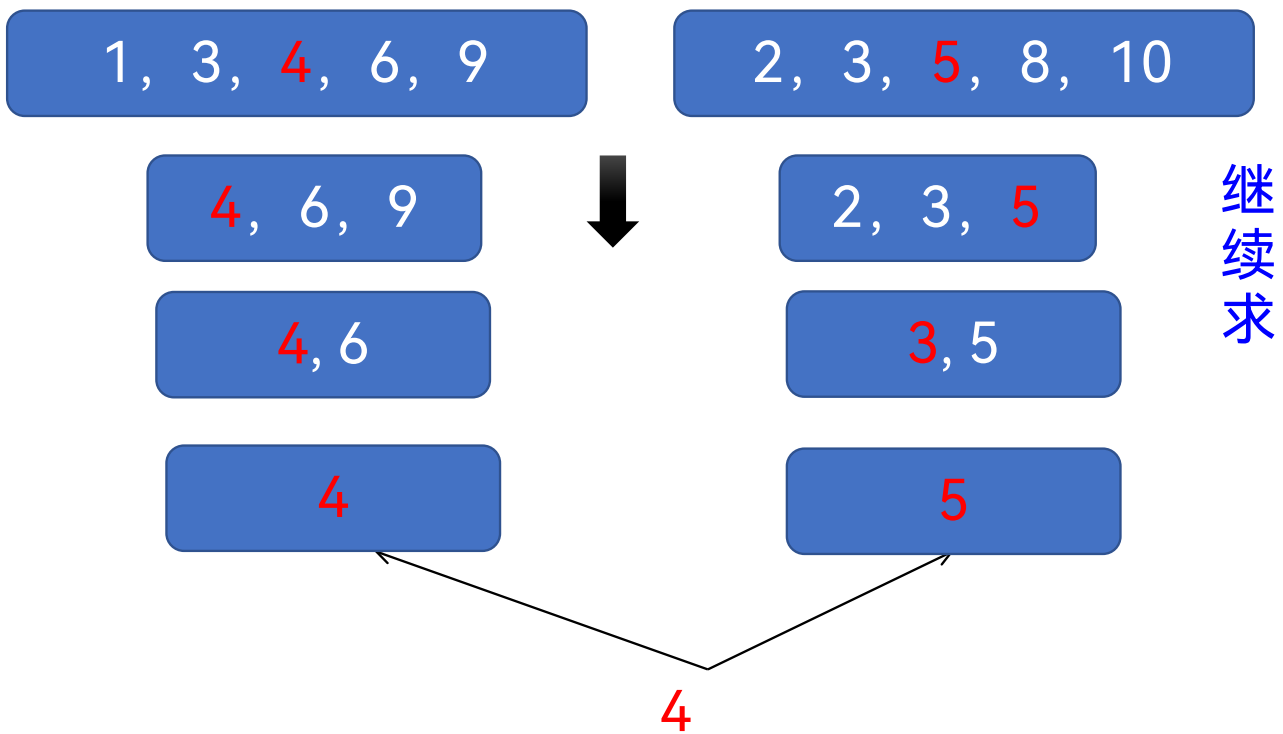
采用二分法求含有n个有序元素的序列a、b的中位数的过程如下：

分别求出a、b的中位数a[m1]和b[m2]：

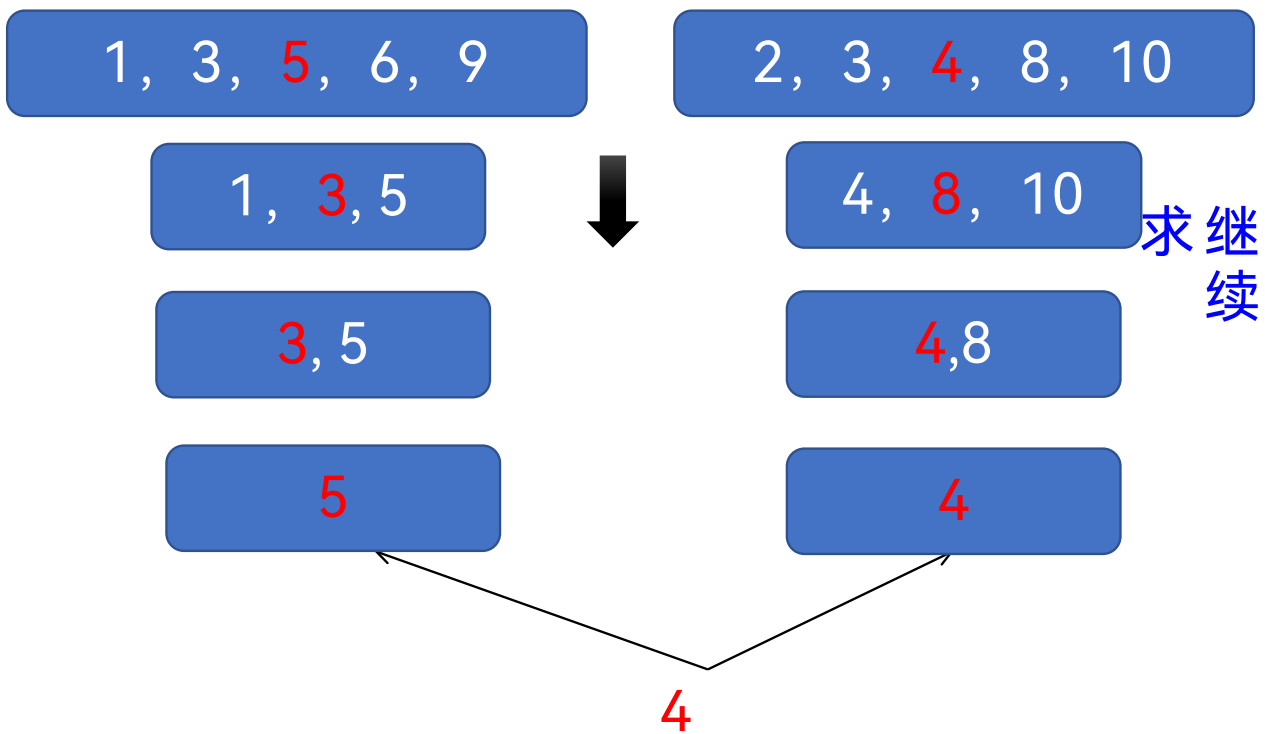
① 若a[m1]=b[m2]，则a[m1]或b[m2]即为所求中位数，算法结束。



② 若*a*[m1]<*b*[m2]，则舍弃序列*a*中前半部分（较小的一半），同时舍弃序列*b*中后半部分（较大的一半）要求舍弃的长度相等。



③ 若*a*[m1]>*b*[m2]，则舍弃序列*a*中后半部分（较大的一半），同时舍弃序列*b*中前半部分（较小的一半），要求舍弃的长度相等。



```
void prepart (int &s, int&t)
{
  int m=(s+t)/2;
   t=m;
}

Void postpart(int &s, int&t)
{
   int m=(s+t)/2;
   if((s+t)%2==0) //序列中有奇数个元素
        s=m;
   else s=m+1;//序列中有偶数个元素
}
```

```
int midnum(int a[]，int s1，int t1，int b[]，int s2，int t2)
{  //求两个有序序列a[s1..t1]和b[s2..t2]的中位数
      int m1，m2;
      if (s1==t1 && s2==t2)     //两序列只有一个元素时返回较小者
            return a[s1]<b[s2]?a[s1]:b[s2];

      else
      {  m1=(s1+t1)/2;          //求a的中位数
         m2=(s2+t2)/2;          //求b的中位数
         if (a[m1]==b[m2])      //两中位数相等时返回该中位数
                  return a[m1];

         if (a[m1]<b[m2])       //当a[m1]<b[m2]时
         {   postpart(s1，t1);//a取后半部分
              prepart(s2，t2); //b取前半部分
               return midnum(a，s1，t1，b，s2，t2);
         }

        else                    //当a[m1]>b[m2]时
        {  prepart(s1，t1);     //a取前半部分
           postpart(s2，t2);  //b取后半部分
           return midnum(a，s1，t1，b，s2，t2);
         }
      }
}
```

【算法分析】对于含有n个元素的有序序列a和b，设调用midnum(a，0，n-1，b，0，n-1)求中位数的执行时间为$T(n)$，显然有以下递归式：

$$T(n)=1 \quad \text{当} n=1$$
$$T(n)=T(n/2)+1 \quad \text{当} n>1$$

容易推出，$T(n)=O(\log_2 n)$。

2.10 求解最大连续子序列和问题

【问题描述】给定一个有n（n≥1）个整数的序列，要求求出其中最大连续子序列的和。

例如：

序列（-2，11，-4，13，-5，-2）的最大子序列和为20

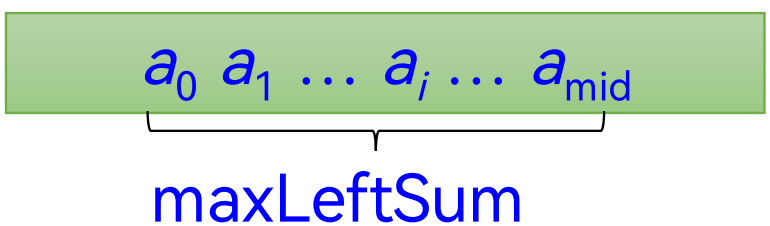
序列（-6，2，4，-7，5，3，2，-1，6，-9，10，-2）的最大子序列和为16。

规定一个序列最大连续子序列和至少是0，如果小于0，其结果为0。

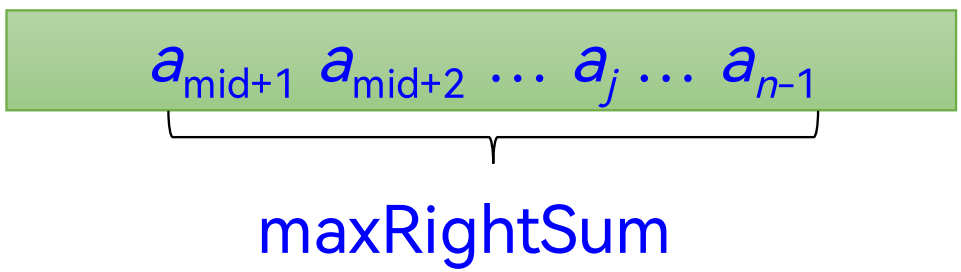
【问题求解】对于含有n个整数的序列$a[0..n-1]$，若$n=1$，表示该序列仅含一个元素，如果该元素大于0，则返回该元素；否则返回0。

若$n>1$，采用分治法求解最大连续子序列时，取其中间位置mid=⌊$(n-1)/2$⌋，该子序列只可能出现3个地方。

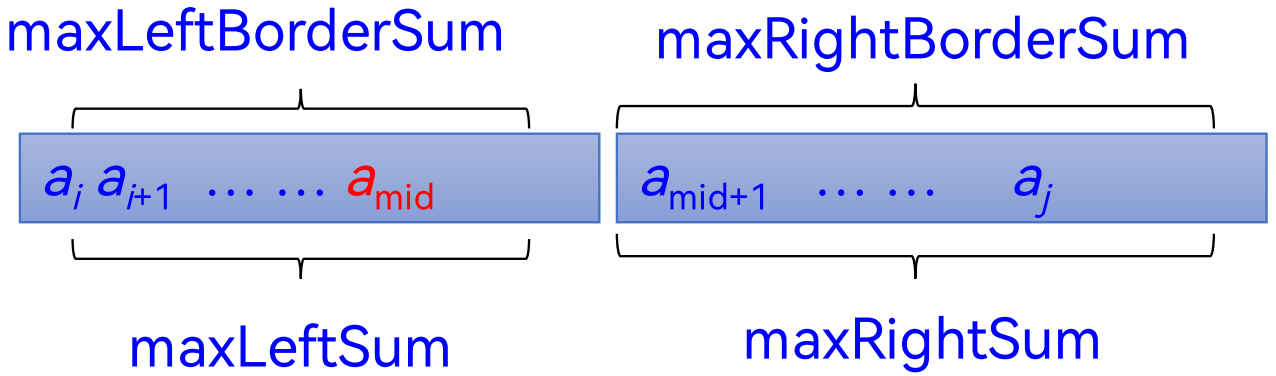
（1）该子序列完全落在左半部即$a[0..mid]$中。采用递归求出其最大连续子序列和maxLeftSum。



（2）该子序列完全落在右半部即a[mid+1..n-1]中。采用递归求出其最大连续子序列和maxRightSum。



（3）该子序列跨越序列*a*的中部而占据左右两部分。



结果：max3(maxLeftSum,
maxRightSum,
maxLeftBorderSum+maxRightBorderSum)

```
long maxSubSum(int a[]，int left，int right)
//求a[left..high]序列中最大连续子序列和
{ int i，j;
  long maxLeftSum，maxRightSum;
  long maxLeftBorderSum，leftBorderSum;
  long maxRightBorderSum，rightBorderSum;

  if (left==right)       //子序列只有一个元素时
  {     if (a[left]>0)  //该元素大于0时返回它
                return a[left];
          else                  //该元素小于或等于0时返回0
                    return 0;
  }
```

```
    int mid=(left+right)/2;                 //求中间位置
    maxLeftSum=maxSubSum(a，left，mid);    //求左边
    maxRightSum=maxSubSum(a，mid+1，right);   //求右边
    maxLeftBorderSum=0，leftBorderSum=0;
    for (i=mid;i>=left;i--)                       //求出以左边加上a[mid]元素
    {  leftBorderSum+=a[i];                       //构成的序列的最大和
     if (leftBorderSum>maxLeftBorderSum)
              maxLeftBorderSum=leftBorderSum;
    }
    maxRightBorderSum=0，rightBorderSum=0;
    for (j=mid+1;j<=right;j++)              //求出a[mid]右边元素
    {  rightBorderSum+=a[j];                //构成的序列的最大和
       if (rightBorderSum>maxRightBorderSum)
              maxRightBorderSum=rightBorderSum;
    }
    return max3(maxLeftSum，maxRightSum，
              maxLeftBorderSum+maxRightBorderSum);
}
```

【算法分析】设求解序列$a[0..n-1]$最大连续子序列和的执行时间为$T(n)$，第（1）、（2）两种情况的执行时间为$T(n/2)$，第（3）种情况的执行时间为$O(n)$，所以得到以下递推式：

$$T(n)=1 \quad \text{当} n=1$$
$$T(n)=2T(n/2)+n \quad \text{当} n>1$$

容易推出，$T(n)=O(n\log_2 n)$。